

Competitive Programming

Section 41: Euler Tour, LCA, and Binary Lifting

Competitive Programming Course

2026-09-13

Table of contents

1	What this section adds	3
2	Euler tour: a subtree is an interval	5
3	LCA by lifting	7
4	Three path queries from one primitive	9
5	Implementation	11
5.1	One DFS, five arrays	11
5.2	Two tables, one loop	12
5.3	The queries	13
5.4	Constraints and language notes	14
5.5	Pitfalls	14
6	Exercises	16

1 What this section adds

Section 38 built the jump table and used it to answer one question: *where do I land after k upward edges?* That question is about a single vertex. This section answers questions about a **path between two vertices**, which is the shape almost every tree problem actually takes.

Everything from Section 38 carries over unchanged:

```
up[k][v] = node reached after exactly  $2^k$  upward edges
up[0][v] = parent[v]
up[k][v] = up[k-1][ up[k-1][v] ]
```

Recall the two conventions from Section 38, because both matter here. The exponent k is a table level, not an edge count: `up[2][v]` means four edges. And node 0 is the sentinel meaning *no such ancestor*; every `up[k][0]` stays 0, so a jump that overshoots the root is harmless.

The new ingredient is that **one DFS produces everything at once**. The same traversal that fills `depth` and `up[0]` can also stamp two timestamps per vertex, and those timestamps answer ancestor questions without any lifting at all. So a single preprocessing pass gives four arrays:

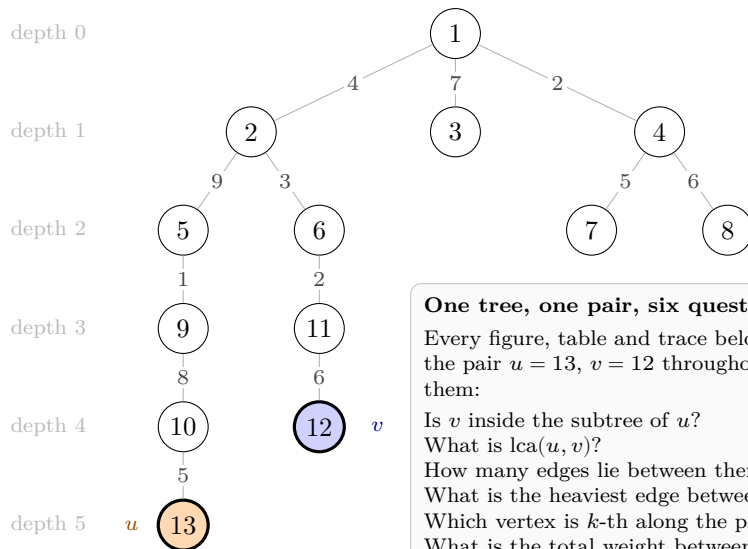
Array	Meaning	Used for
<code>depth[v]</code>	edges from the root to v	equalizing depths, distance
<code>up[0][v]</code>	parent of v	seeding the jump table
<code>tin[v], tout[v]</code>	entry and exit clock	subtree tests, subtree ranges

One term runs through every chapter, so it is worth fixing before anything uses it. The **lowest common ancestor** of u and v , written $\text{lca}(u, v)$, is the deepest vertex that is an ancestor of both. Equivalently: walk from u up to the root and from v up to the root, and take the first vertex the two walks have in common.

A vertex counts as an ancestor of itself. That convention is not decoration — it is what makes $\text{lca}(u, v) = v$ the right answer when v already lies on the root path of u , and it is why the algorithm below can stop early when the two vertices become equal.

Here is the tree that everything below is built on. It is rooted at 1, and its edges carry weights that the next three chapters ignore and the last one uses. A single pair, $u = 13$ and $v = 12$, runs through every figure, table and trace, so each picture builds on the one before it.

Read the six questions in the box as the plan for the section. The first is answered by timestamps alone and needs no table at all. The second is the lowest common ancestor just defined, answered by the jump table. The remaining four all reduce to the second, which is why it is worth building carefully before anything else.



One tree, one pair, six questions.

Every figure, table and trace below uses this rooted, edge-weighted tree, and the pair $u = 13$, $v = 12$ throughout. The questions, in the order we answer them:

- | | |
|---|---------------------|
| Is v inside the subtree of u ? | Euler tour |
| What is $\text{lca}(u, v)$? | lifting, two phases |
| How many edges lie between them? | problem A |
| What is the heaviest edge between them? | problem B |
| Which vertex is k -th along the path? | problem C |
| What is the total weight between them? | problem D |

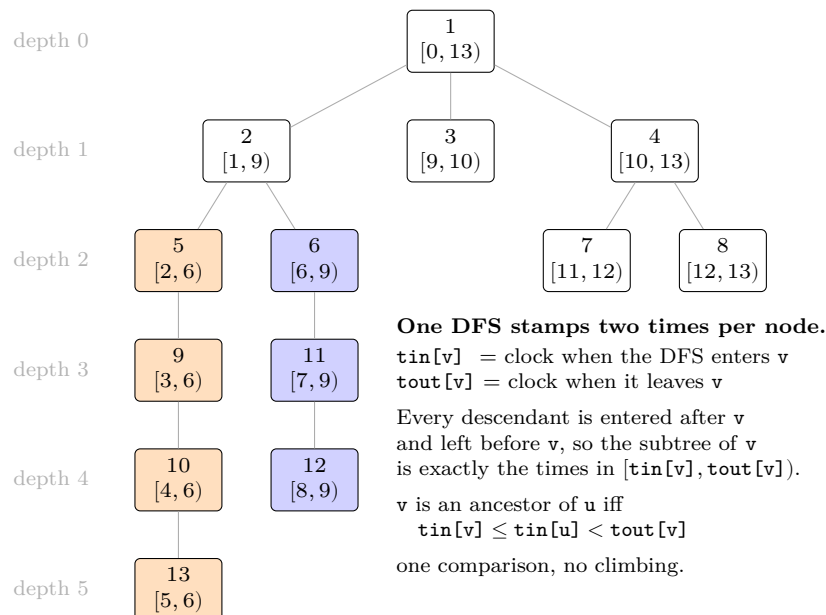
Edge labels are weights; they are ignored until the weighted questions.

2 Euler tour: a subtree is an interval

Run a DFS from the root and keep a single clock. On entering v , record $\text{tin}[v]$ and tick the clock. On leaving v , record $\text{tout}[v]$ without ticking.

```
tin[v] = clock value when the DFS enters v
tout[v] = clock value when the DFS leaves v
```

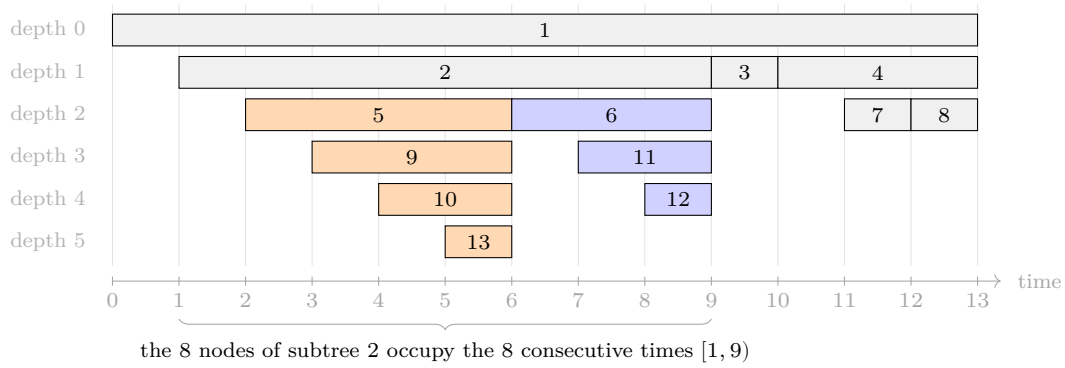
Because a DFS finishes a whole subtree before it returns, every descendant of v is entered after v and left before v :



That containment is the whole idea, and it gives an ancestor test with no climbing and no table:

```
v is an ancestor of u <=> tin[v] <= tin[u] < tout[v]
```

Laying the intervals on a time axis makes the nesting explicit. Sibling subtrees occupy disjoint, adjacent blocks; a parent's block exactly spans its children's blocks plus its own entry tick:



Reading the interval $[\text{tin}[v], \text{tout}[v])$ as a range is what turns subtree questions into array questions. If you write every vertex into an array at position $\text{tin}[v]$, the subtree of v becomes a contiguous slice, so “add to every node in a subtree” or “sum a subtree” becomes a range update or range query on a Fenwick or segment tree from Sections 36 and 37. Section 42 leans on this directly.

Two conventions are in circulation, and mixing them is an easy off-by-one. This section stores $\text{tout}[v]$ as the clock value on *leaving* v , which is one past the last time in the subtree, so the interval is half-open and the ancestor test ends in a strict $<$. Others store $\text{tout}[v]$ as the last time *inside* the subtree and write the closed interval $[\text{tin}[v], \text{tout}[v]]$ with a $\leq$ test; Section 42 uses that closed form. Both describe the same set of vertices. Pick one per solution and keep it.

For the running tree the stamps are:

v	1	2	3	4	5	6	7	8	9	10	11	12	13
depth	0	1	1	1	2	2	2	2	3	4	3	4	5
tin	0	1	9	10	2	6	11	12	3	4	7	8	5
tout	13	9	10	13	6	9	12	13	6	6	9	9	6

Check one row against the picture: $\text{tin}[2] = 1$ and $\text{tout}[2] = 9$, an interval of length 8, and the subtree of 2 is $\{2, 5, 6, 9, 10, 11, 12, 13\}$ — exactly 8 vertices. The length of the interval is the size of the subtree.

An ancestor test is worth having even when you also have the jump table. $\text{lca}(u, v) == u$ costs $O(\log n)$; $\text{tin}[u] \leq \text{tin}[v] < \text{tout}[u]$ costs one comparison.

3 LCA by lifting

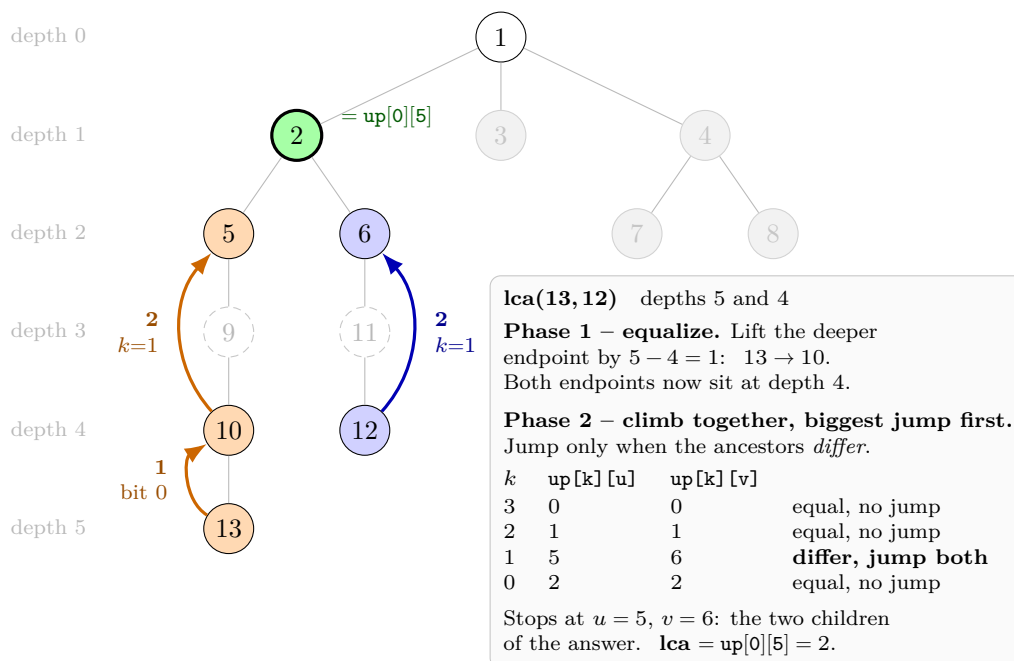
The jump table answers the path questions through one primitive: the lowest common ancestor. The algorithm has two phases, and the second one is where the subtlety lives.

1. Make u the deeper vertex.
2. **Equalize.** Lift u by $\text{depth}[u] - \text{depth}[v]$, using the bits of that difference exactly as in Section 38.
3. If $u == v$, then v was an ancestor of u and it is the answer.
4. **Climb together.** Working from the largest level down to zero, lift **both** vertices by 2^k , but only when $\text{up}[k][u] \neq \text{up}[k][v]$.
5. The two vertices now sit immediately below the answer, so return $\text{up}[0][u]$.

One word of warning about the usual name for step 4. Both vertices only ever move **upward** in it; nothing descends the tree. What counts downward is k , the loop over jump sizes, from $2^{(\text{LOG}-1)}$ down to 2^0 . The step is often called *descending the table* for exactly that reason, and the phrase trips people up: it is the jump sizes that descend, never the vertices.

Here is the full table for the running tree, followed by a complete trace:

v	1	2	3	4	5	6	7	8	9	10	11	12	13
$\text{up}[0]$	0	1	1	1	2	2	4	4	5	9	6	11	10
$\text{up}[1]$	0	0	0	0	1	1	1	1	2	5	2	6	9
$\text{up}[2]$	0	0	0	0	0	0	0	0	0	1	0	1	2
$\text{up}[3]$	0	0	0	0	0	0	0	0	0	0	0	0	0



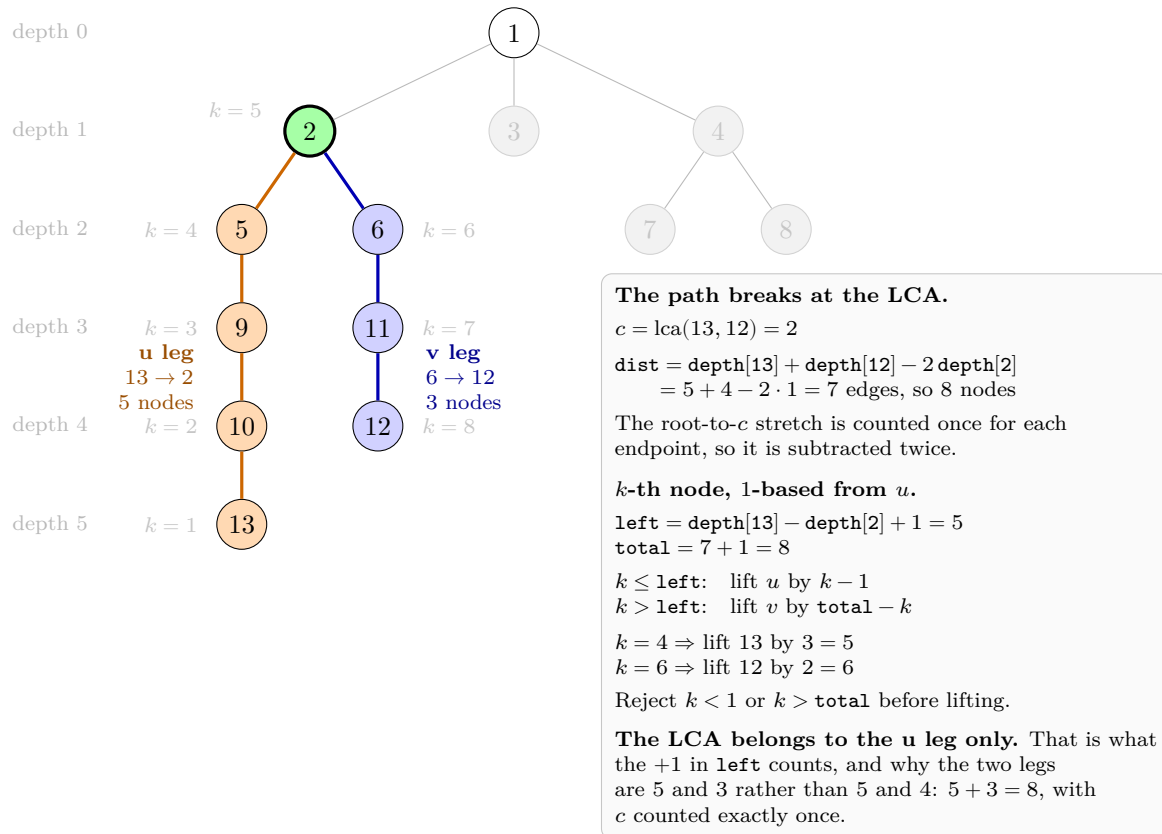
Why the comparison comes before the jump. Step 4 tests $\text{up}[k][u]$ against $\text{up}[k][v]$ and moves only when they differ. If the two ancestors are *equal*, the jump would land on or above the LCA, and from there the two vertices could never be separated again. Jumping only on a difference keeps both vertices strictly below the LCA at every moment, so the loop can never overshoot.

Read the trace above in that light. At $k = 1$ the ancestors are 5 and 6, which differ, so both move — they are still below the answer. At $k = 0$ the ancestors are both 2, so the loop refuses to move, and 2 is precisely the answer. The final $\text{up}[0][u]$ is not an extra step; it is the one jump the loop deliberately declined to make.

Note also that $\text{up}[2][10] = \text{up}[2][12] = 1$ did not trigger a jump even though 1 is a genuine common ancestor. That is the same rule working correctly: 1 is common but not *lowest*, and equality is exactly the signal to stay put.

4 Three path queries from one primitive

Once $c = \text{lca}(u, v)$ is known, the path from u to v splits into two disjoint upward legs, $u \rightarrow c$ and $c \leftarrow v$. Every path query is a statement about those two legs.



Distance. The two legs share only c , and the root-to- c stretch is counted once inside $\text{depth}[u]$ and once inside $\text{depth}[v]$, so it is removed twice:

$$\text{distance}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}(u, v)]$$

That formula counts **edges**. The number of vertices on the path is one more. Confusing the two is the most common wrong answer in this section, and $u == v$ is the test that catches it: zero edges, one vertex.

K-th node. The same split decides which leg holds the answer, as annotated in the figure above. A k counted from u either lands on the u leg, where it is a plain lift, or on the v leg, where it is easier to count *upward from v* than downward from c :

```

left  = depth[u] - depth[c] + 1      (vertices on the u leg, u..c)
total = depth[u] + depth[v] - 2*depth[c] + 1

k <= left : answer = lift(u, k - 1)
k > left  : answer = lift(v, total - k)

```

The reverse lift is what avoids materializing the path as a list. Reject $k < 1$ or $k > \text{total}$ before lifting, or the sentinel will quietly return 0.

Maximum edge. Store a second table with the same shape, where $\text{mx}[k][v]$ is the heaviest edge on the 2^k -edge upward run starting at v :

```

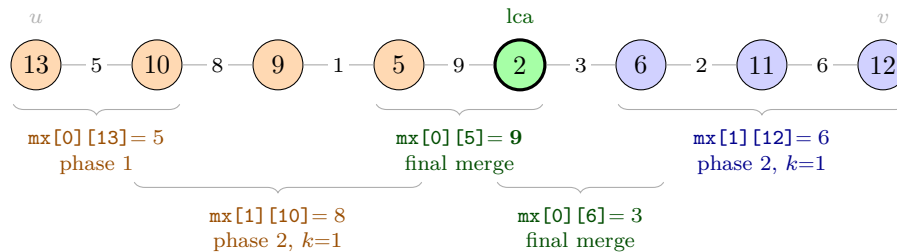
mx[0][v] = weight of the edge from v to its parent
mx[k][v] = max( mx[k-1][v], mx[k-1][ up[k-1][v] ] )

```

v	1	2	3	4	5	6	7	8	9	10	11	12	13
mx[0]	0	4	7	2	9	3	5	6	1	8	2	6	5
mx[1]	0	4	7	2	9	4	5	6	9	8	3	6	8
mx[2]	0	4	7	2	9	4	5	6	9	9	4	6	9

Entries whose jump overshoots the root — $\text{mx}[1][2]$, for instance — describe a truncated run and are meaningless, but they are never read: phase 1 only uses bits of a real depth difference, and phase 2 only jumps when the ancestors differ, which keeps both vertices below the LCA.

Now run the LCA exactly as before, but merge the corresponding mx entry into the answer *before* each move. The lifts partition the path into disjoint chunks, each of which is one table entry:



Every one of the seven edges lies in exactly one chunk, and each chunk is a single table entry, so the answer is one maximum over five lookups: $\max(5, 8, 9, 3, 6) = 9$.

The trap. The heaviest edge 9 sits in a *final-merge* chunk, below the last jump. Code that returns $\text{up}[0][u]$ without first merging $\text{mx}[0][u]$ and $\text{mx}[0][v]$ prints 8. Always merge the aggregate *before* overwriting the vertex.

Weighted distance is the same table shape with a different merge. Because sums telescope, it does not even need a per-chunk merge: keep $\text{sw}[v]$, the summed weight from the root to v , in the same DFS, and reuse the distance identity.

```

weighted_distance(u,v) = sw[u] + sw[v] - 2*sw[lca(u,v)]

```

For $u = 13, v = 12$: $\text{sw}[13] = 27, \text{sw}[12] = 15, \text{sw}[2] = 4$, giving $27 + 15 - 8 = 34$, which matches the edge weights along the path, $5+8+1+9+3+2+6 = 34$. Maximum does not telescope — there is no inverse — which is exactly why it needs the mx table while sums do not.

5 Implementation

Everything in this section comes from one traversal and two tables. This chapter writes them out in full; the code below is the code that produced every number in the figures.

5.1 One DFS, five arrays

Recursion is not an option at $n = 2e5$: a path-shaped tree overflows the C++ stack and blows Python's recursion limit. The iterative version pushes every vertex twice — once to enter it, once to leave it — which is exactly what gives `tout` its meaning.

```
int n, LOG, timer = 0;
vector<vector<pair<int,int>>> adj;    // (neighbour, edge weight)
vector<int> par, wpar;              // parent, and its edge weight
vector<int> dep, tin, tout;         // depth, plus the Euler stamps
vector<long long> sw;               // weighted depth from the root

void root_tree(int r) {
    // stack holds (vertex, 0 = entering, 1 = leaving)
    vector<pair<int,int>> st{{r, 0}};
    par[r] = 0;
    while (!st.empty()) {
        auto [v, leaving] = st.back();
        st.pop_back();
        if (leaving) { tout[v] = timer; continue; }
        tin[v] = timer++;
        st.push_back({v, 1});          // schedule the exit stamp
        for (auto it = adj[v].rbegin(); it != adj[v].rend(); ++it) {
            auto [c, w] = *it;         // reversed: pops in order
            if (c == par[v]) continue; // the only edge back up
            par[c] = v;
            wpar[c] = w;
            dep[c] = dep[v] + 1;
            sw[c] = sw[v] + w;
            st.push_back({c, 0});
        }
    }
}
```

```
def root_tree(r):
    timer = 0
    st = [(r, 0)]                    # (vertex, 0 enter, 1 leave)
    par[r] = 0
    while st:
```

```

v, leaving = st.pop()
if leaving:
    tout[v] = timer
    continue
tin[v] = timer
timer += 1
st.append((v, 1))          # schedule the exit stamp
for c, w in reversed(adj[v]): # the stack pops them in order
    if c == par[v]:
        continue
    par[c], wpar[c] = v, w
    dep[c] = dep[v] + 1
    sw[c] = sw[v] + w
    st.append((c, 0))

```

The exit stamp is pushed *before* the children, so it is popped *after* all of them — that is the entire trick, and it is why `tout[v]` ends up one past the last time in the subtree.

A stack reverses whatever you push, so the children are pushed in reverse to come back out in ascending order — which is what makes the stamps above match the figures exactly. Any child order is equally correct; only the numeric stamps change, never the containment property, and no query depends on it.

If the input is a parent list rather than an edge list, `par`, `wpar` and `dep` can be filled while reading, and this DFS is still needed only if you want the Euler stamps.

5.2 Two tables, one loop

```

vector<vector<int>> up;          // up[k][v]: 2^k-th ancestor
vector<vector<long long>> mx;    // mx[k][v]: heaviest edge there

LOG = 1;
while ((1 << LOG) <= n + 1) LOG++;
up.assign(LOG, vector<int>(n + 1, 0)); // 0 = the sentinel
mx.assign(LOG, vector<long long>(n + 1, 0));

for (int v = 1; v <= n; v++) { up[0][v] = par[v]; mx[0][v] = wpar[v]; }

for (int k = 1; k < LOG; k++)
    for (int v = 1; v <= n; v++) {
        int m = up[k - 1][v];          // midpoint of the 2^k jump
        up[k][v] = up[k - 1][m];
        mx[k][v] = max(mx[k - 1][v], mx[k - 1][m]);
    }

```

Naming the midpoint is worth the extra line: `up[k-1][v]` is read twice, once as the destination of the first half-jump and once as the start of the second.

5.3 The queries

```
bool is_ancestor(int u, int v) {           // no lifting at all
    return tin[u] <= tin[v] && tin[v] < tout[u];
}

int lift(int v, int d) {
    for (int k = 0; d; k++, d >>= 1)
        if (d & 1) v = up[k][v];
    return v;
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    u = lift(u, dep[u] - dep[v]);           // phase 1: equalize
    if (u == v) return u;                   // v was an ancestor
    for (int k = LOG - 1; k >= 0; k--)      // phase 2: climb both
        if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
    return up[0][u];                       // the jump we declined
}
```

The two dist queries need no new code:

```
int dist(int u, int v) {
    int c = lca(u, v);
    return dep[u] + dep[v] - 2 * dep[c];
}

long long wdist(int u, int v) {
    int c = lca(u, v);
    return sw[u] + sw[v] - 2 * sw[c];
}
```

The maximum-edge query is the same walk with one **max** merged in **before** every move:

```
long long max_edge(int u, int v) {
    long long best = 0;
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int k = 0; d; k++, d >>= 1)
        if (d & 1) { best = max(best, mx[k][u]); u = up[k][u]; }
    if (u == v) return best;
    for (int k = LOG - 1; k >= 0; k--)
        if (up[k][u] != up[k][v]) {
            best = max({best, mx[k][u], mx[k][v]});
            u = up[k][u]; v = up[k][v];
        }
    return max({best, mx[0][u], mx[0][v]}); // below the LCA
}
```

That last line is the one the figure warned about. Delete it and the running example prints 8 instead of 9.

```
int kth(int u, int v, int k) {
    int c = lca(u, v);
    int left = dep[u] - dep[c] + 1;
    int total = dep[u] + dep[v] - 2 * dep[c] + 1;
    if (k < 1 || k > total) return -1; // reject before lifting
    return k <= left ? lift(u, k - 1) : lift(v, total - k);
}
```

In Python the tables and queries translate line for line; only `lca` is shown, since it is the one with the two phases:

```
def lca(u, v):
    if dep[u] < dep[v]:
        u, v = v, u
    d = dep[u] - dep[v] # phase 1: equalize
    k = 0
    while d:
        if d & 1:
            u = up[k][u]
        d >>= 1
        k += 1
    if u == v:
        return u
    for k in range(LOG - 1, -1, -1): # phase 2: climb both
        if up[k][u] != up[k][v]:
            u, v = up[k][u], up[k][v]
    return up[0][u]
```

5.4 Constraints and language notes

With n, q near $2e5$, preprocessing is $O(n \log n)$ and each query is $O(\log n)$. $LOG = \text{ceil}(\log_2(n+1))$ is the smallest safe number of levels.

C++. Value initialization already sets row 0 of `up` to the sentinel, so there is nothing to clear. Use `long long` for `sw` and for `mx` if weights can sum — weighted depth on a path of $2e5$ maximal weights overflows `int` easily. Initialize an empty maximum to 0 only when weights are known to be nonnegative; otherwise use `LLONG_MIN`.

Python. Bind the table rows to locals inside the build loop (`upk, upp = up[k], up[k-1]`) before iterating; attribute and index lookups dominate otherwise. Read input with `sys.stdin.buffer.read().split()`. $LOG = (n+1).bit_length()$ is clear and safe.

5.5 Pitfalls

Mistake	Test that catches it
confusing edges and vertices in distance	<code>u == v</code> must print zero
omitting the equalize phase	an ancestor-descendant pair
jumping when ancestors are equal	any query whose LCA is the root
merging the aggregate after moving the vertex	one heavy edge just below the LCA
dropping the final <code>mx[0]</code> merge	the running example prints 8, not 9
treating a 1-based <code>k</code> as 0-based	the first and last vertex on a path
forgetting to reject out-of-range <code>k</code>	<code>k = 0</code> and <code>k = total + 1</code>
recursive DFS	a path-shaped tree of <code>2e5</code> vertices
<code>int</code> overflow on weighted sums	a deep path of maximal weights

Run the local reference suite with:

```
CP_TARGET=solution \
python3 sections/41_euler_tour_lca_binary_lifting/check.py
```

For trees of at most 20 vertices, a parent-walk oracle can enumerate each path directly and check distance, maximum edge, weighted distance, and every valid and invalid `k`.

6 Exercises

Use A to separate edge counts from vertex counts in the distance formula, B to place the aggregate merge before the vertex move, and C to split a path at the LCA and count from the correct end. Use D as the blind synthesis problem: decide what a lift must accumulate when the answer is a sum rather than a maximum, and whether the `mx` table is needed at all. Test `u == v`, an ancestor-descendant pair, a query whose LCA is the root, the first and last `k` on a path, and a `k` one past the end.

- A. Tree Distances: Given a rooted tree and queries `u v`, print the number of edges on the path.
- B. Max Edge On Path: Each edge to a parent has a weight. For each `u v`, print the maximum edge weight on the path.
- C. Kth Node On Path: For query `u v k`, print the `k`-th node on the path from `u` to `v`, or `-1`.
- D. Weighted Tree Distances: For each `u v`, print the sum of edge weights on the path.